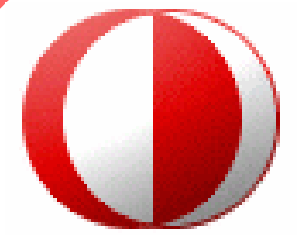


FastSLAM

A Feasible Particle Filter based approach

Department of Electrical and Electronics Engineering
Dr. Afşar Saranlı

Lecture slides heavily use material from the textbook and
Sebastian Thrun, Lecture Slides; http://www.probabilistic-robotics.org/

What we will discuss

- Some refreshers,
- Why SLAM is difficult with Particle Filters,
- An important assumption,
- Factorization leading to a feasible particle model,
- FastSLAM 1.0 (known data association & Landmarks),
- Unknown Data association,
- Occupancy Grid Based FastSLAM
- Examples
- Ideas for FastSLAM 2.0 (and when it is better),
- Summary

Remember the SLAM Problem

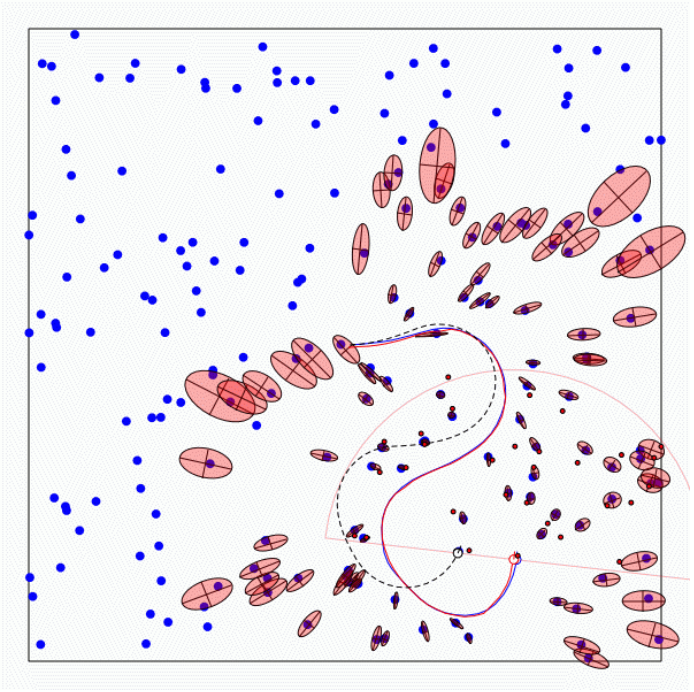
- **“Simultaneous Localization and Mapping”**
- **The task of building a map while estimating the pose of the robot relative to this map.**

Given:

- The robot’s controls
- Observations of nearby features

Estimate:

- Map of features
- Path of the robot



Remember EKF SLAM

- **"Augment" the system state (robot pose) by the static parameters of the map:**
- **For landmark based maps, State becomes:**

$$<(x, y, \theta), (l_x^1, l_y^1), (l_x^2, l_y^2), ..., (l_x^N, l_y^N)>$$

- **Dimension: (3+2*_N_),**
- **In EKF-SLAM:**
  posterior distribution* or *belief
  → a multivariate gaussian function
- **Representation scales well to higher dimensions,**
- **However, update complexity grows *quadratically* with the dimension of the problem.**

Remember Particle Filters

- Represent posterior (belief) by random samples,
- Capable of estimating non-Gaussian and/or nonlinear processes,
- Sampling Importance resampling (SIR) principle,
  - Generate *predicted* particles from *motion model,*
  - Assign *importance weights* to particles using measurement model,
  - *Resample particles* to generate new particle set.
- Typical applications upto now: Tracking, Localization (low dimensional !!)

Localization versus SLAM

- **Localization state space:** $<(x, y, \theta)>$
- **SLAM state space (landmark based):**

$$<(x, y, \theta), (l_x^1, l_y^1), (l_x^2, l_y^2), \ldots, (l_x^N, l_y^N)>$$

- **SLAM state space (grid based):**

$$<(x, y, \theta), c_{11}, c_{12}, \ldots, c_{1n}, c_{21}, \ldots, c_{mn}>$$

- **A particle filter could have been used to solve both problems… But: We have a problem!!**
- **Problem: The number of particles needed to represent the posterior grows *exponentially* with the *dimension of the state space*.**

Difficulty of SLAM w Particle Filters

- **A naïve implementation with particle filters…**

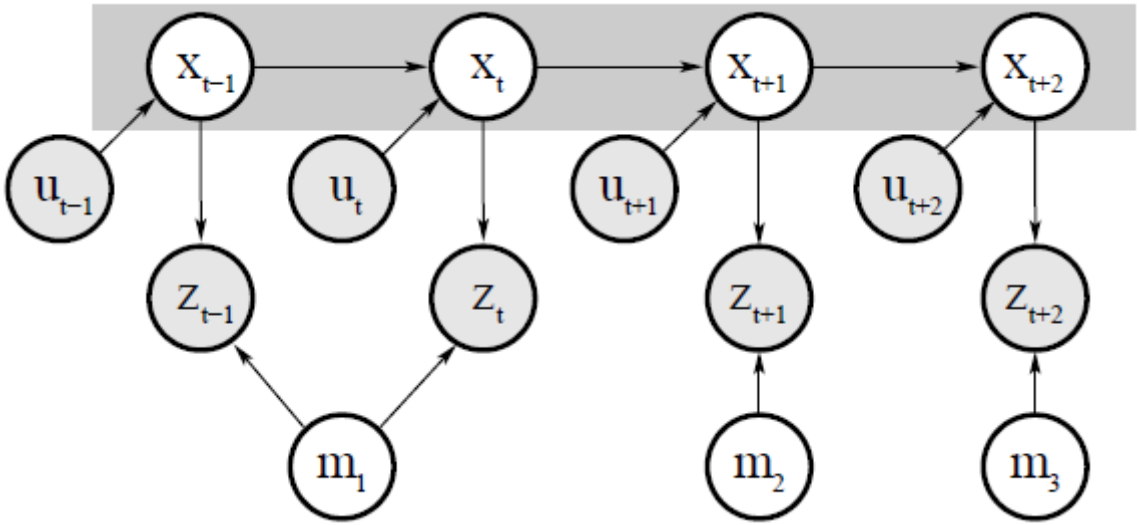
is doomed to Fail due to
Curse of Dimensionality

Important Observation

- A joint state space is needed for estimation because of the correlations between components,
- For uncorrelated entries or “entry blocks” of the state space: Representation can be decoupled:
- Posterior represented by a product of densities
- Can we find such uncorrelated blocks in the SLAM problem?

Important Observation

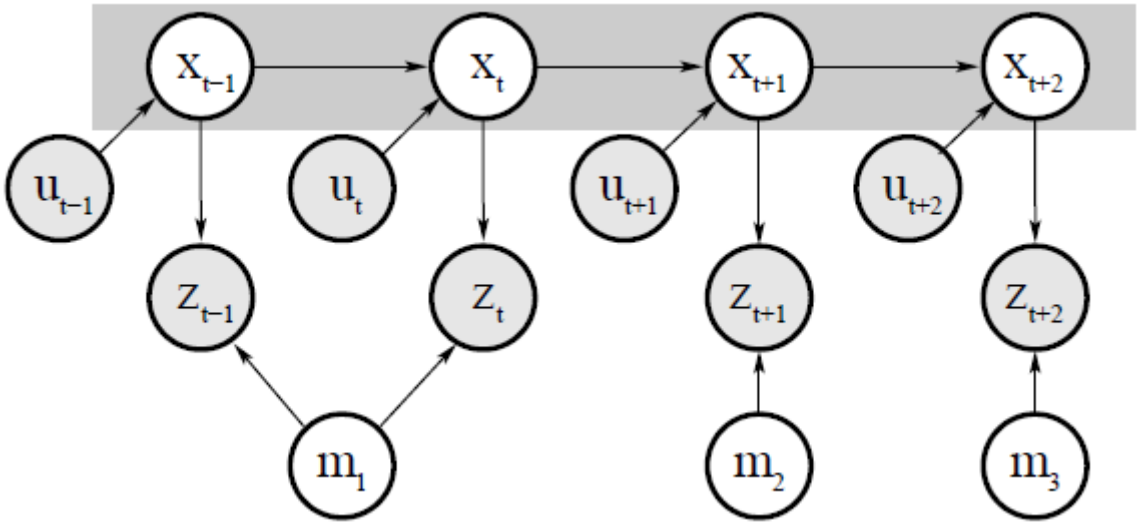
- **Hmmm….**



- **Map estimates are correlated with the robot pose *because of relative measurements*,**
- **Pose variables are correlated with each other because of motion,**
- **Individual Landmark estimates are correlated with each other because of the *robot pose they are measured relative to…***

Important Assumption

- **But the map feature estimates would be independent if we knew the robot pose history,**



- **What does it mean graphically on the Bayes Net?**

Factoring the SLAM Posterior

- **"Rao-Blackwellization": A useful factorization**
- **Consider:**

$$P(a, b) = p(b|a) \cdot p(a)$$

- **If $p(b|a)$ can be computed in closed form, represent only $p(a)$ with samples and for each such sample, compute (or "estimate") $p(b|a)$**

Factoring the SLAM Posterior

- **"Rao-Blackwellization"**
- **It allows us to write:**

$$p(x_{1:t}, l_{1:N} \mid z_{1:t}, u_{1:t}) =$$
$$p(x_{1:t} \mid z_{1:t}, u_{1:t}) \cdot p(l_{1:N} \mid x_{1:t}, z_{1:t})$$

Use $P(a, b|c) = P(a|c) \cdot P(b|a.c)$ where we have

$$a = x_{1:t}$$
$$b = l_{1:N}$$
$$c = z_{1:t}, u_{1:t}$$

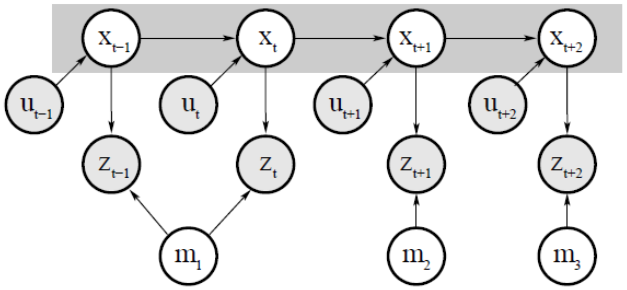
and the "conditional independence that states:

$$p(l_{1:N}|x_{1:t}, z_{1:t}, u_{1:t}) = p(l_{1:N}|x_{1:t}, z_{1:t})$$

because we cannot gain any extra information from $u_{1:t}$ if we know $x_{1:t}$

Factoring the SLAM Posterior

- **"Rao-Blackwellization".**
- **It allows us to write:**



$$p(x_{1:t}, l_{1:N} \mid z_{1:t}, u_{1:t}) =$$

$$p(x_{1:t} \mid z_{1:t}, u_{1:t}) \cdot p(l_{1:N} \mid x_{1:t}, z_{1:t})$$

- **And then…**

$$= p(x_{1:t} \mid z_{1:t}, u_{1:t}) \cdot \prod_{n=1}^{N} p(l_n \mid x_{1:t}, z_{1:t})$$

Robot path posterior (Localization problem)

Conditionally Independent Landmark Positions (map)

Factoring the SLAM Posterior

- **More informally:**

  "If someone told us the robot path, the landmark positions would be conditionally independent!!"

- **more applicable/implementable version:**

  "if we sampled the robot path, for each such sample, we can represent landmark positions as N independent estimates"

- **Even more implementable version:**

  "use a particle filter to estimate the robot path, then use *N* separate Gaussian Filters for each particle to estimate individual landmark positions"

Wow!

Particle Representation

- **This results in a feasible particle representation:**

	robot path	feature 1	feature 2	…	feature N
Particle $k=1$	$x_{1:t}^{[1]} = \{(x \;\; y \;\; \theta)^T\}_{1:t}^{[1]}$	$\mu_1^{[1]}, \Sigma_1^{[1]}$	$\mu_2^{[1]}, \Sigma_2^{[1]}$	…	$\mu_N^{[1]}, \Sigma_N^{[1]}$
Particle $k=2$	$x_{1:t}^{[2]} = \{(x \;\; y \;\; \theta)^T\}_{1:t}^{[2]}$	$\mu_1^{[2]}, \Sigma_1^{[2]}$	$\mu_2^{[2]}, \Sigma_2^{[2]}$	…	$\mu_N^{[2]}, \Sigma_N^{[2]}$
⋮					
Particle $k=M$	$x_{1:t}^{[M]} = \{(x \;\; y \;\; \theta)^T\}_{1:t}^{[M]}$	$\mu_1^{[M]}, \Sigma_1^{[M]}$	$\mu_2^{[M]}, \Sigma_2^{[M]}$	…	$\mu_N^{[M]}, \Sigma_N^{[M]}$

FastSLAM Algorithm

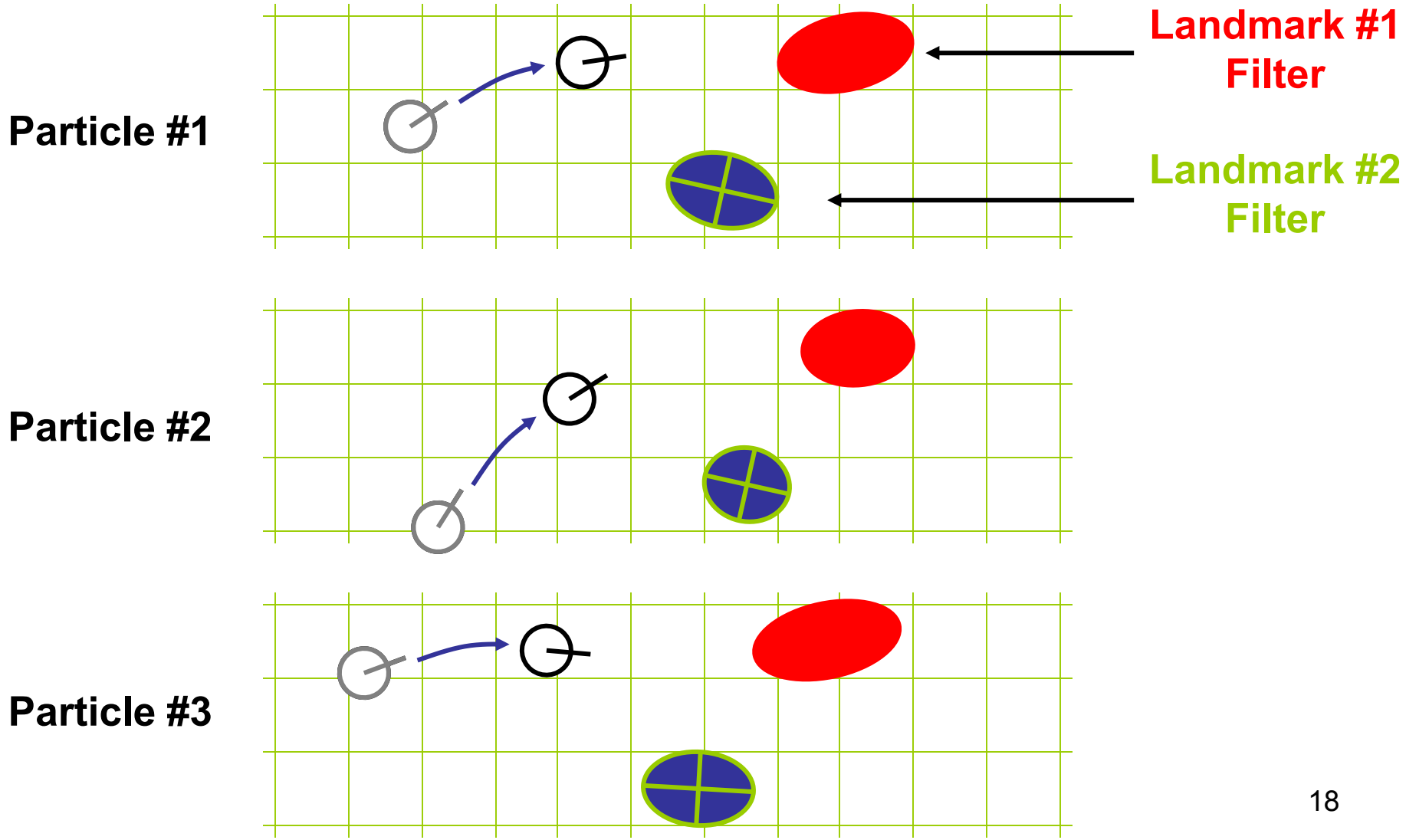
	robot path	feature 1	feature 2	…	feature N
Particle $k=1$	$x_{1:t}^{[1]} = \{(x\ \ y\ \ \theta)^T\}_{1:t}^{[1]}$	$\mu_1^{[1]}, \Sigma_1^{[1]}$	$\mu_2^{[1]}, \Sigma_2^{[1]}$	…	$\mu_N^{[1]}, \Sigma_N^{[1]}$
Particle $k=2$	$x_{1:t}^{[2]} = \{(x\ \ y\ \ \theta)^T\}_{1:t}^{[2]}$	$\mu_1^{[2]}, \Sigma_1^{[2]}$	$\mu_2^{[2]}, \Sigma_2^{[2]}$	…	$\mu_N^{[2]}, \Sigma_N^{[2]}$
⋮					
Particle $k=M$	$x_{1:t}^{[M]} = \{(x\ \ y\ \ \theta)^T\}_{1:t}^{[M]}$	$\mu_1^{[M]}, \Sigma_1^{[M]}$	$\mu_2^{[M]}, \Sigma_2^{[M]}$	…	$\mu_N^{[M]}, \Sigma_N^{[M]}$

- **Rao-Blackwellized particle filtering based on landmarks [Montemerlo, 2002],**
- **Each landmark is estimated by a 2D EKF,**
- **Thus each particle needs to maintain *N* EKFs…**

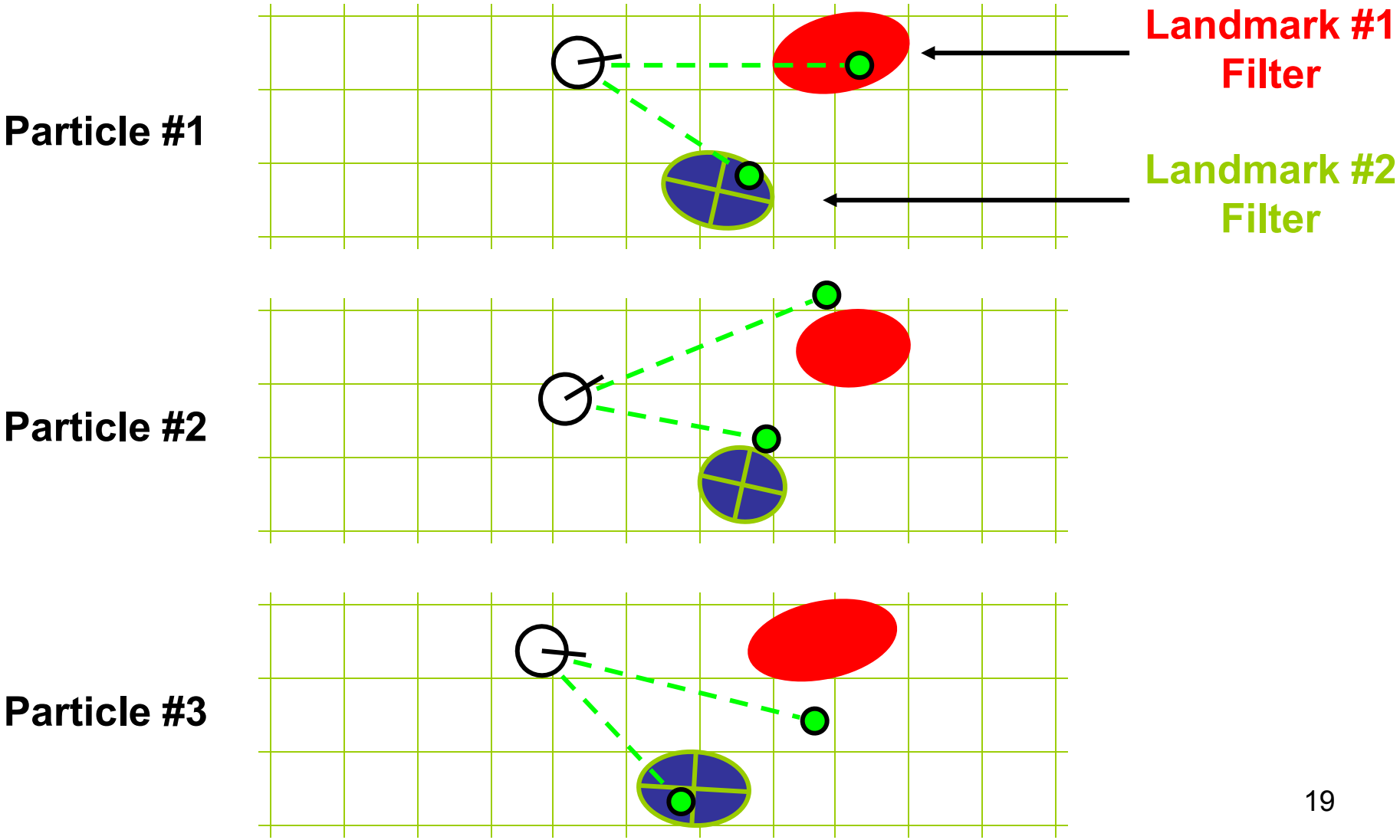
FastSLAM Algorithm

- Do the following M times:
  - **Retrieval.** Retrieve a pose $x_{t-1}^{[k]}$ from the particle set Y_{t-1}.
  - **Prediction.** Sample a new pose $x_t^{[k]} \sim p(x_t \mid x_{t-1}^{[k]}, u_t)$.
  - **Measurement update.** For each observed feature z_t^i identify the correspondence j for the measurement z_t^i, and incorporate the measurement z_t^i into the corresponding EKF, by updating the mean $\mu_{j,t}^{[k]}$ and covariance $\Sigma_{j,t}^{[k]}$.
  - **Importance weight.** Calculate the importance weight $w^{[k]}$ for the new particle.
- **Resampling.** Sample, with replacement, M particles, where each particle is sampled with a probability proportional to $w^{[k]}$.

FastSLAM – Prediction Step

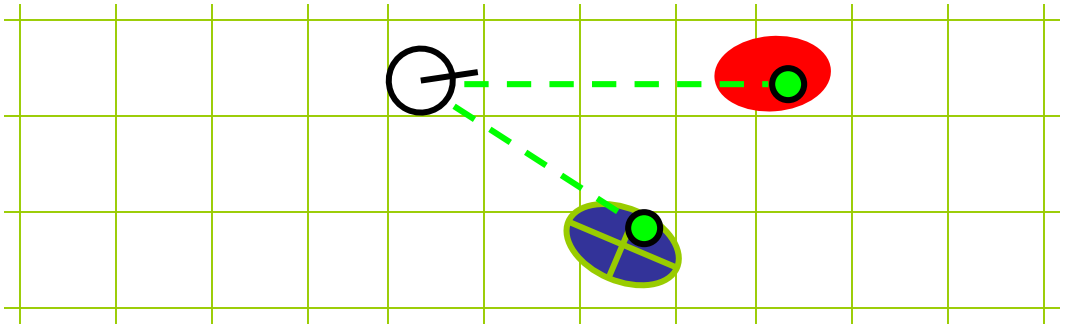


FastSLAM – Measurement Update



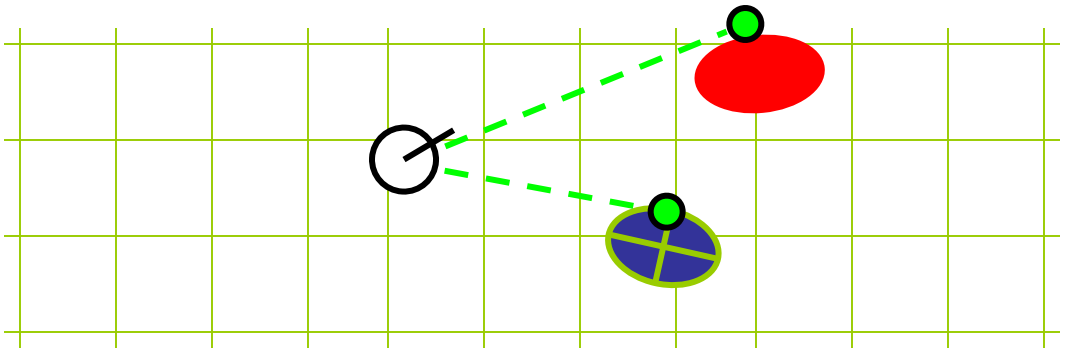
FastSLAM – Measurement Update

Particle #1



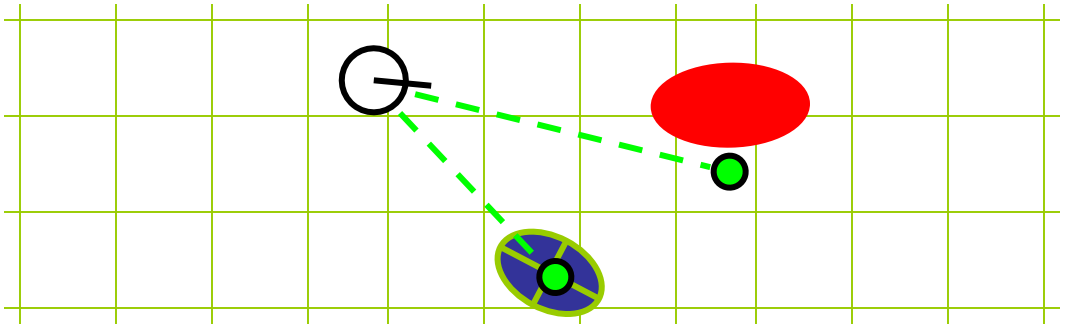
Weight = 0.8

Particle #2



Weight = 0.4

Particle #3



Weight = 0.1

FastSLAM – Complexity

- **Update robot particles based on control u_{t-1}**

  $O(M)$
  Constant time per particle

- **Incorporate observation z_t into Kalman filters**

  $O(M \bullet \log(N))$
  Log time per particle

- **Resample particle set**

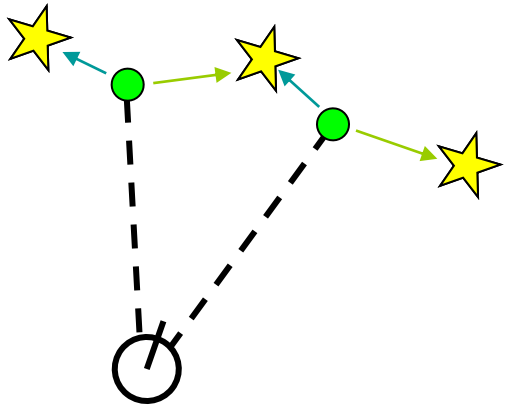
  $O(M \bullet \log(N))$
  Log time per particle

$O(M \bullet \log(N))$
Log time per particle

M **= Number of particles**
N **= Number of map features**

Data Association Problem

- **Which observation belongs to which landmark?**

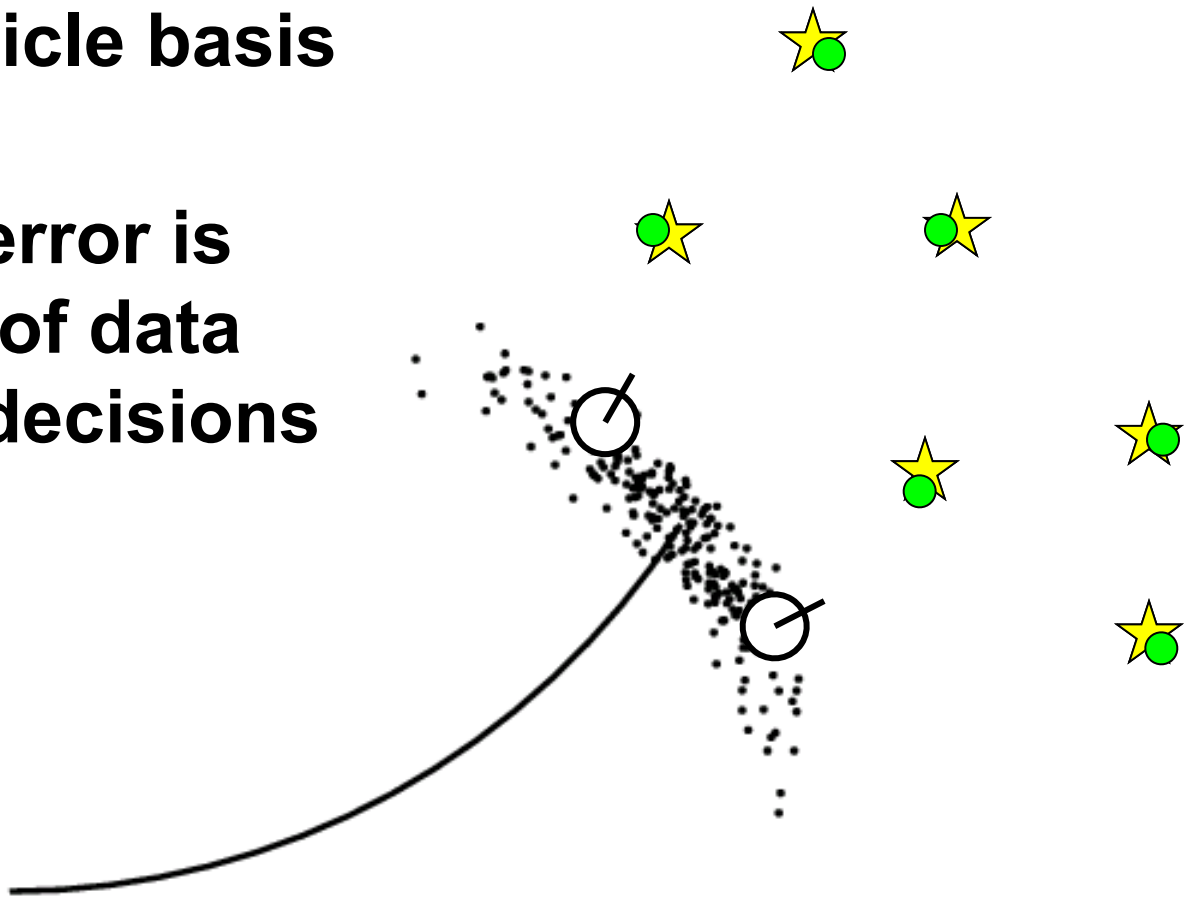


- **A robust SLAM must consider possible data associations**
- **Potential data associations depend also on the pose of the robot**

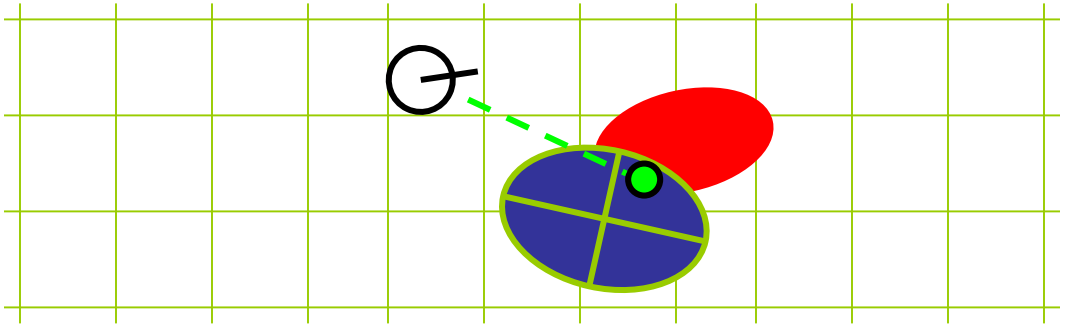
Multi-Hypothesis Data Association

- Data association is done on a per-particle basis

- Robot pose error is factored out of data association decisions



Per-Particle Data Association



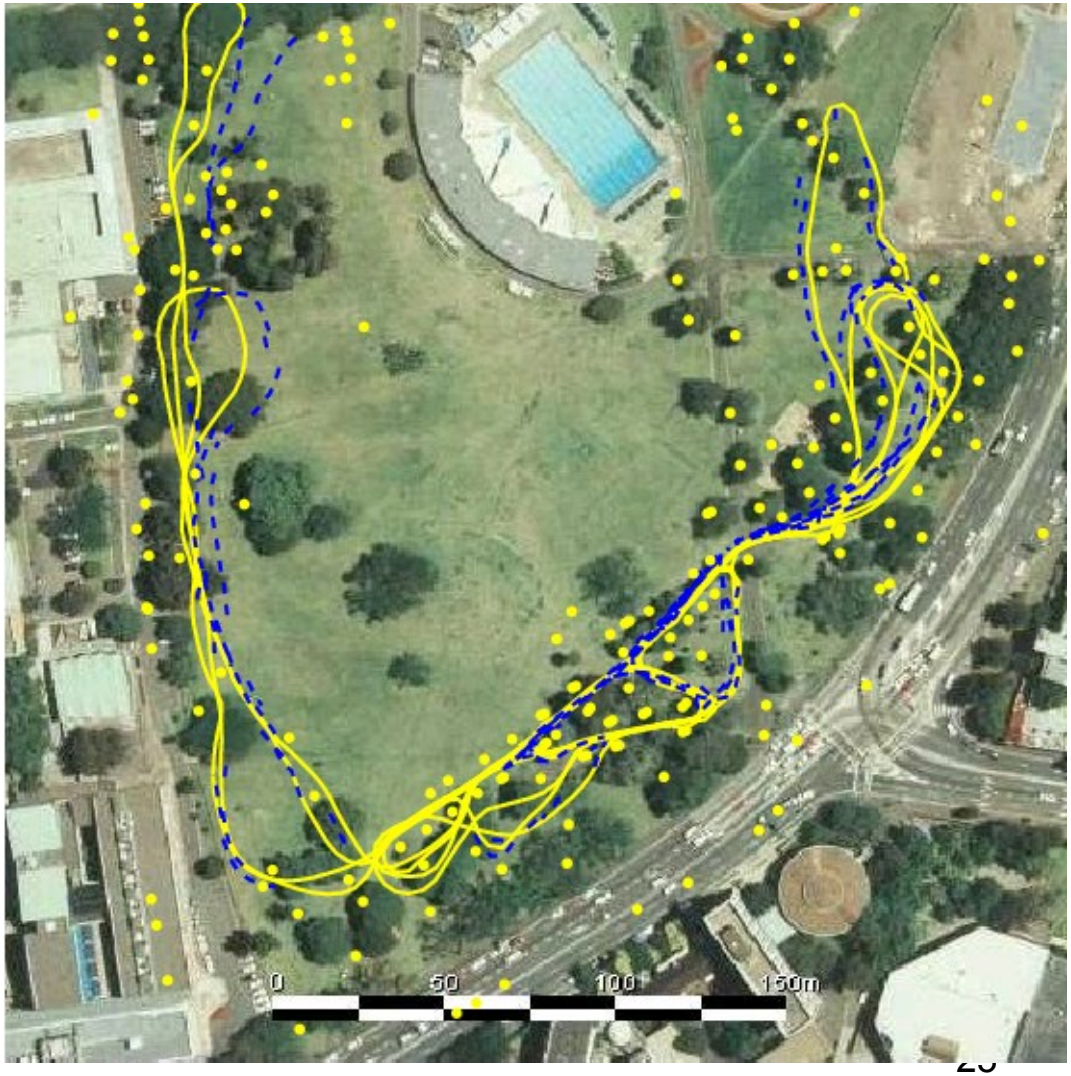
Was the observation generated by the red or the blue landmark?

P(observation|red) = 0.3 P(observation|blue) = 0.7

- **Two options for per-particle data association**
  - **Pick the most probable match**
  - **Pick a random association weighted by the observation likelihoods**
- **If the probability is too low, *generate a new landmark***

Example – Victoria Park Dataset

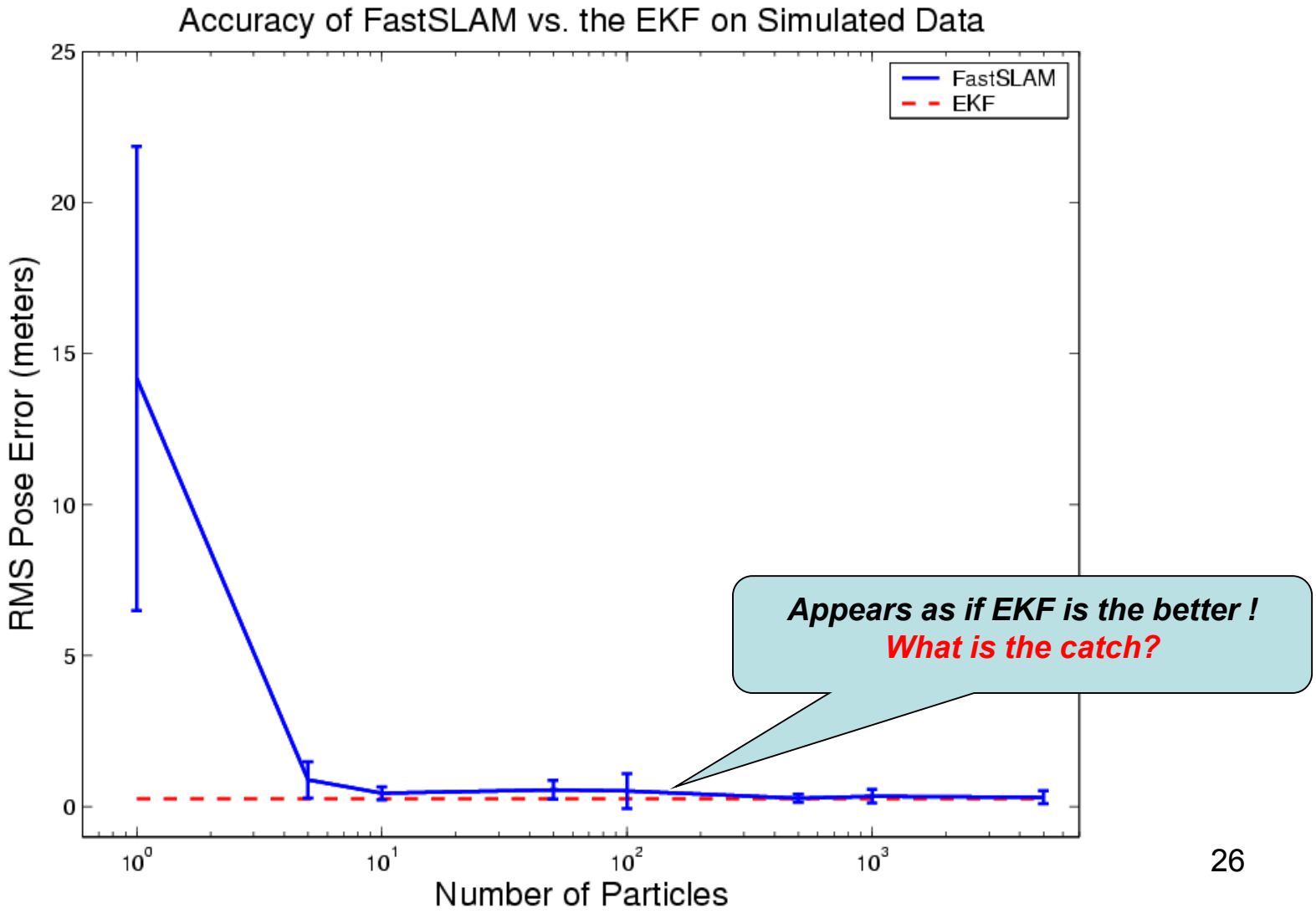
- 4 km traverse
- < 5 m RMS position error
- 100 particles



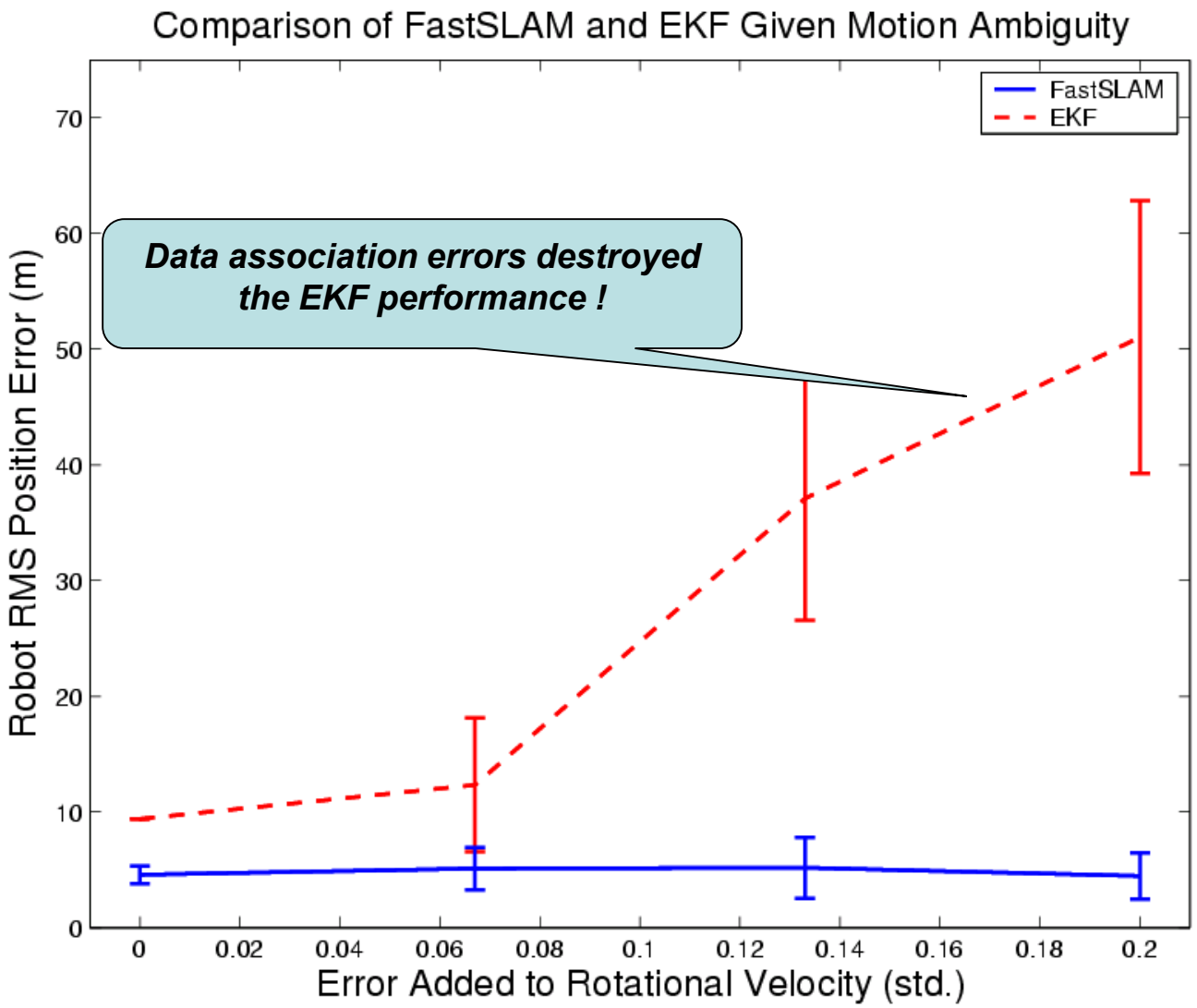
Blue = GPS
Yellow = FastSLAM

Dataset courtesy of University of Sydney

Simulation Results - Accuracy



Simulated – Data Association Effects

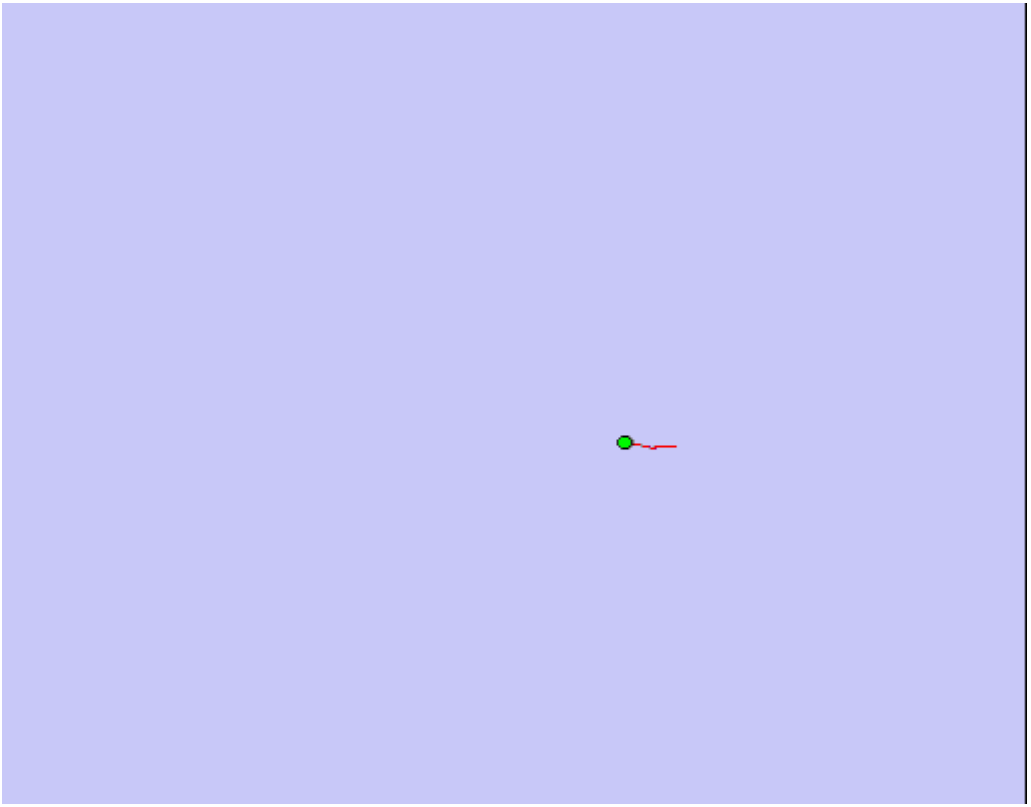


Occupancy Grid Based FastSLAM

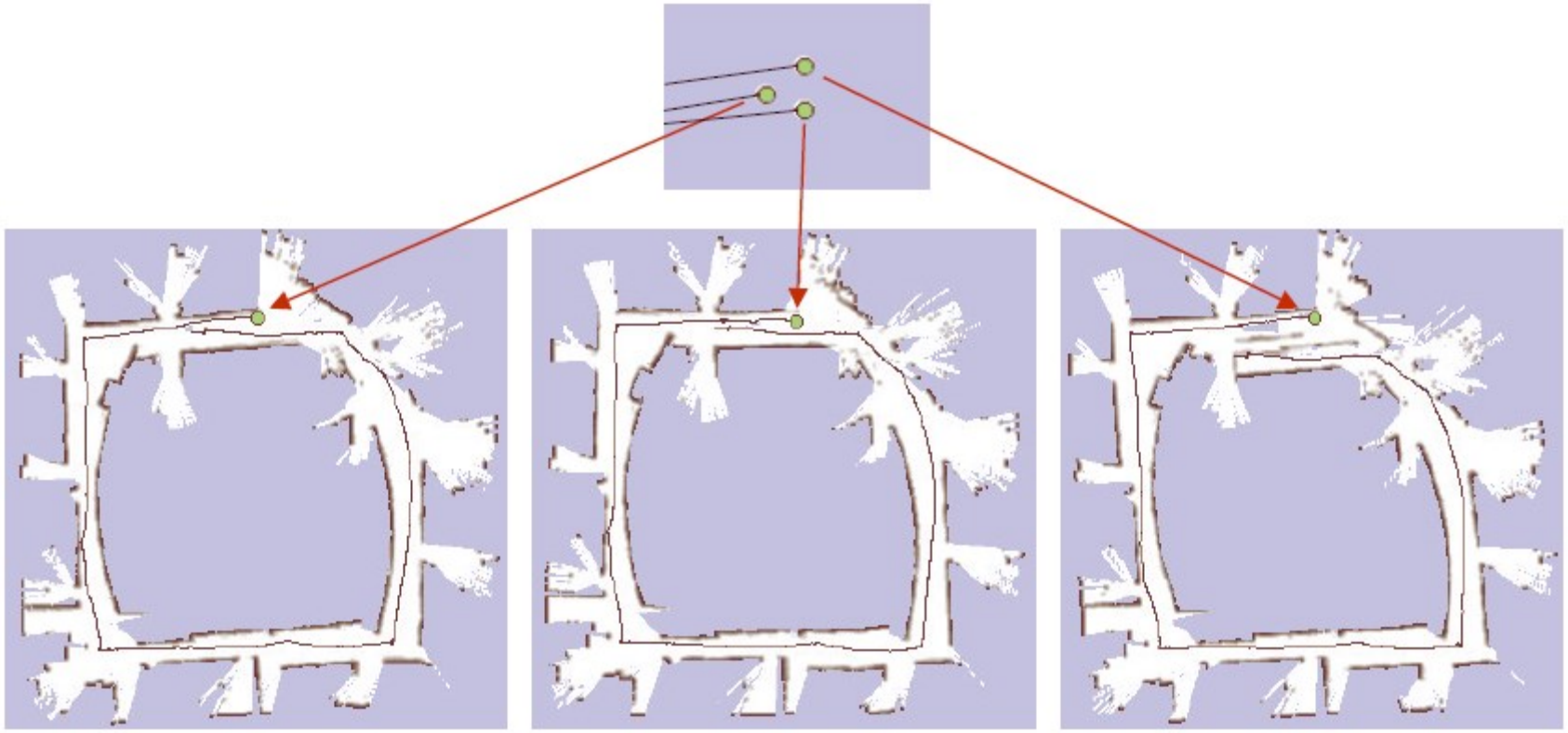
- **Can we solve the SLAM problem if no pre-defined landmarks are available?**
- **Can we use the ideas of FastSLAM to build occupancy grid maps?**
- **As with landmarks, the map depends on the poses of the robot during measurements**
- **If the poses are known, occupancy grid-based mapping is easy ("*mapping with known poses*")**

Mapping with Raw Odometry

- Mapping of the Intel Campus…
- Just to remember what happens…



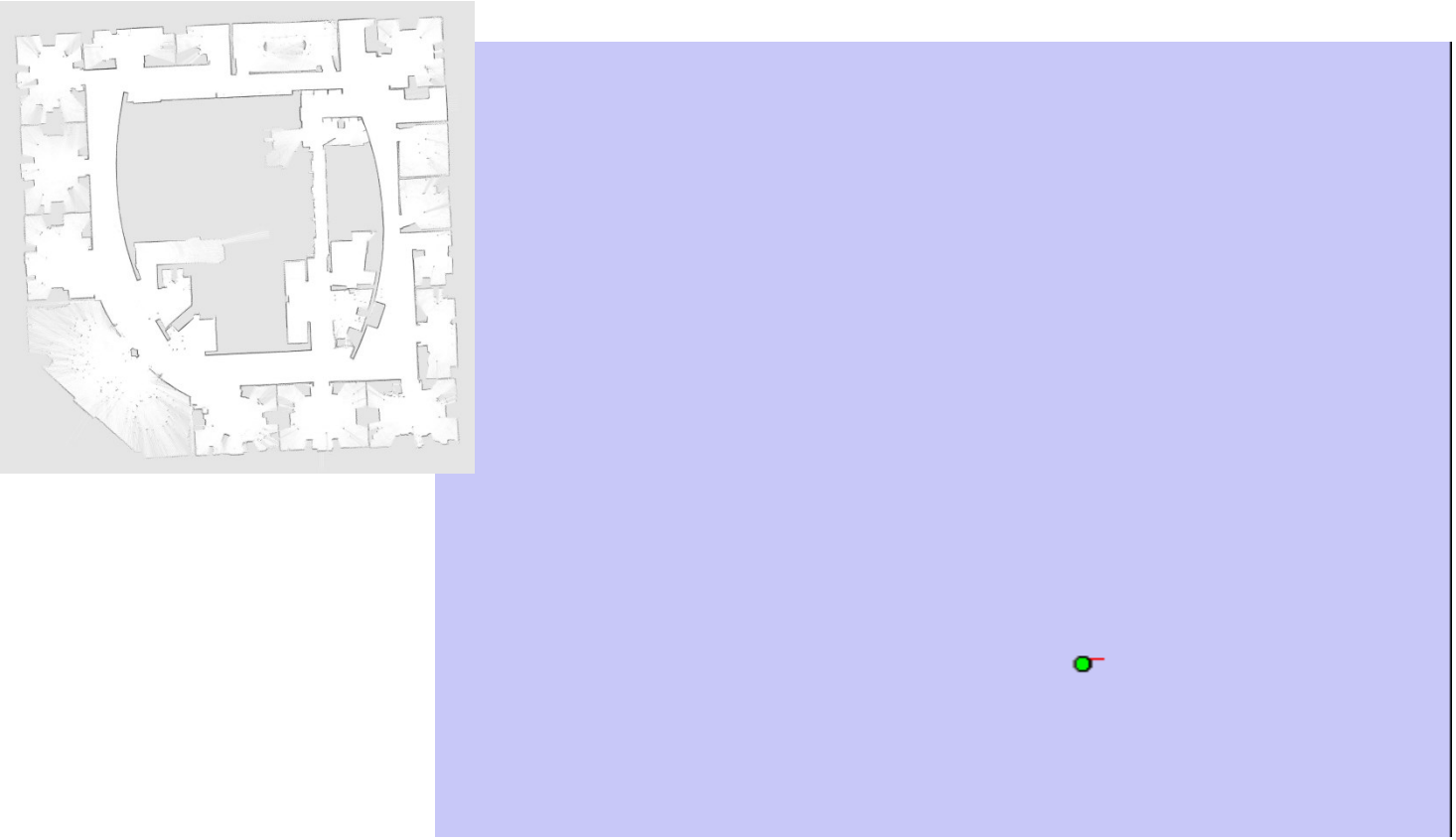
Particle Filter – Particle Examples



- Each particle maintains its own *path* and *map estimate*,
- Weights can be computed by *scan matching*

FastSLAM – Scan Matching

- **Intel Campus – FastSLAM with Scan Matching**



- **Occupancy grid map of the particle with the highest accumulated weight is shown**

Problems with grid-based SLAM

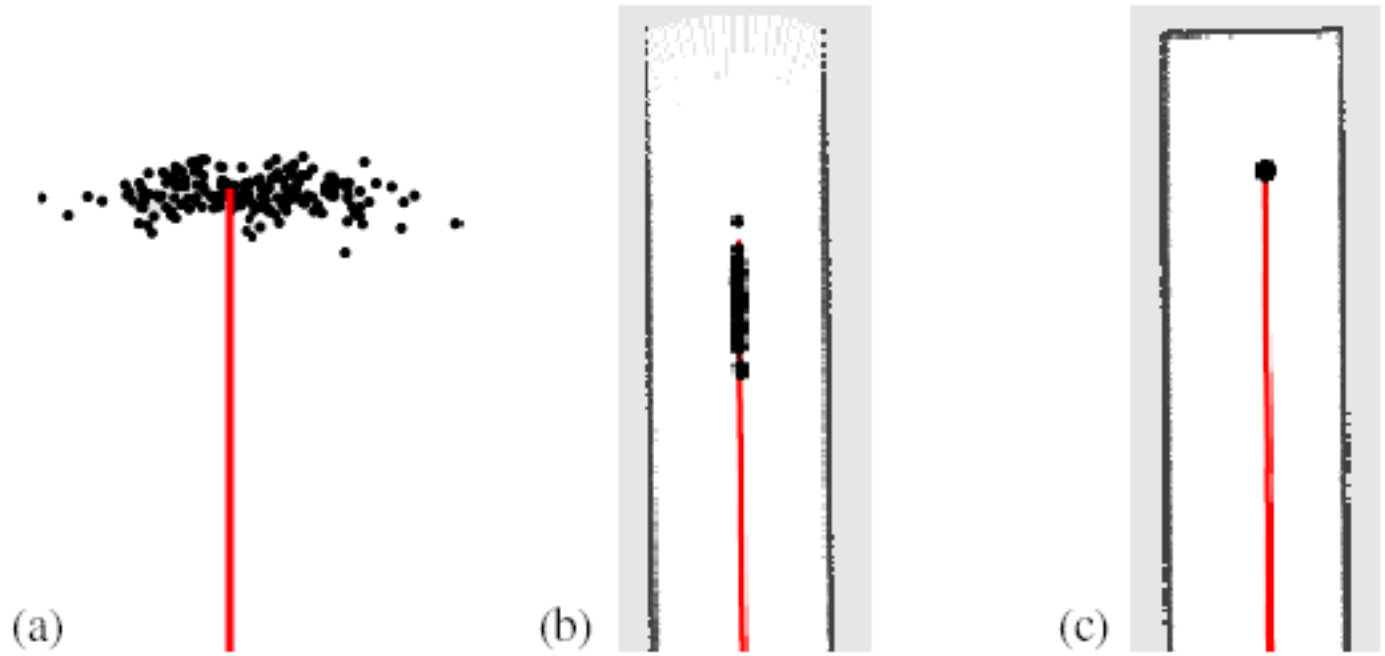
- Each map is quite big in case of grid maps,
- Each particle maintains its own map:
- Thus: one needs to keep the number of particles small… how?
- **One Partial Solution:**
  Compute better proposal distributions!
- **Idea:**
  Improve the pose estimate by using the measurement before applying the particle filter
- (This results in FastSLAM v2.0)

Further Improvements

- **Improved proposal distributions will lead to more accurate maps,**
- **They can be achieved by adapting the proposal distribution according to the most recent observations**
- **Flexible (selective) re-sampling steps can further improve the accuracy.**

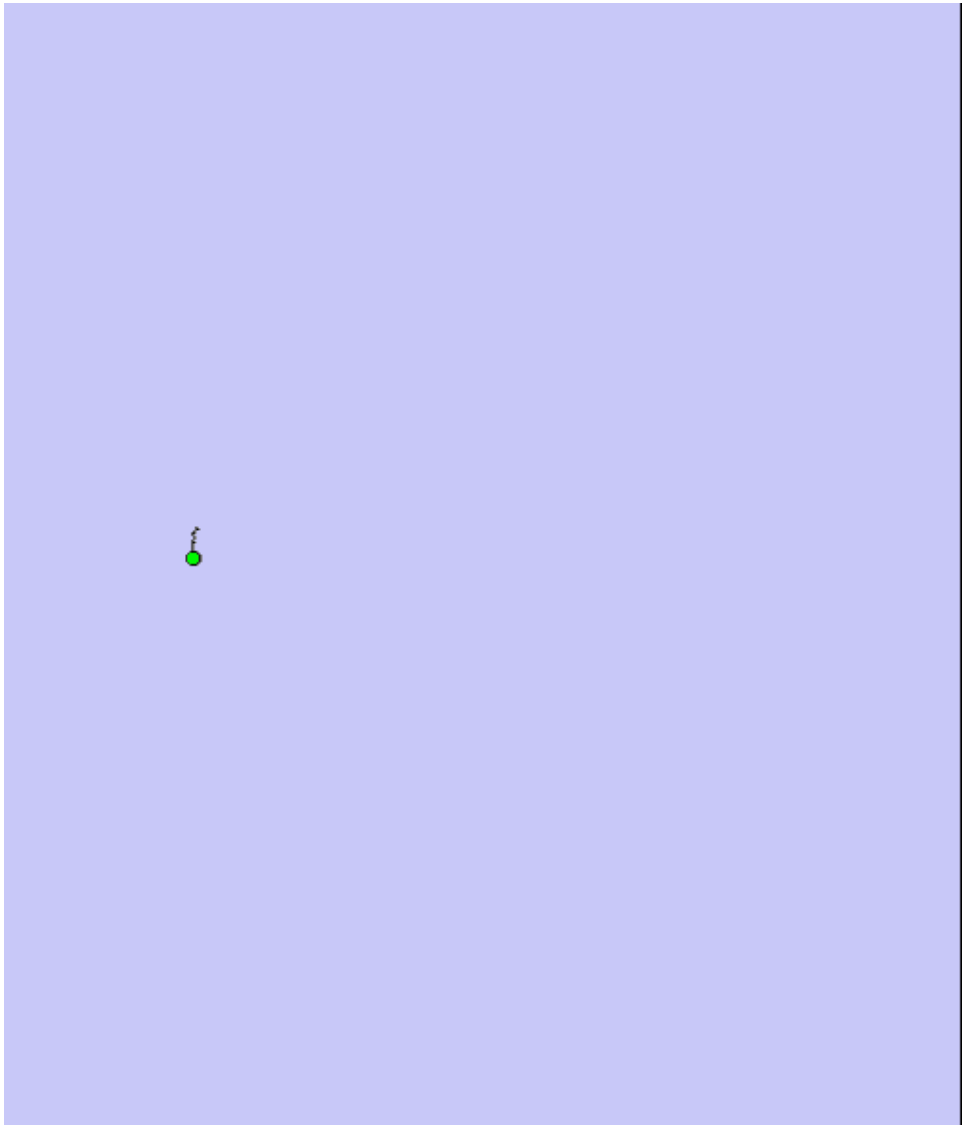
Improved Proposal Example

- **The proposal adapts to the structure of the environment**



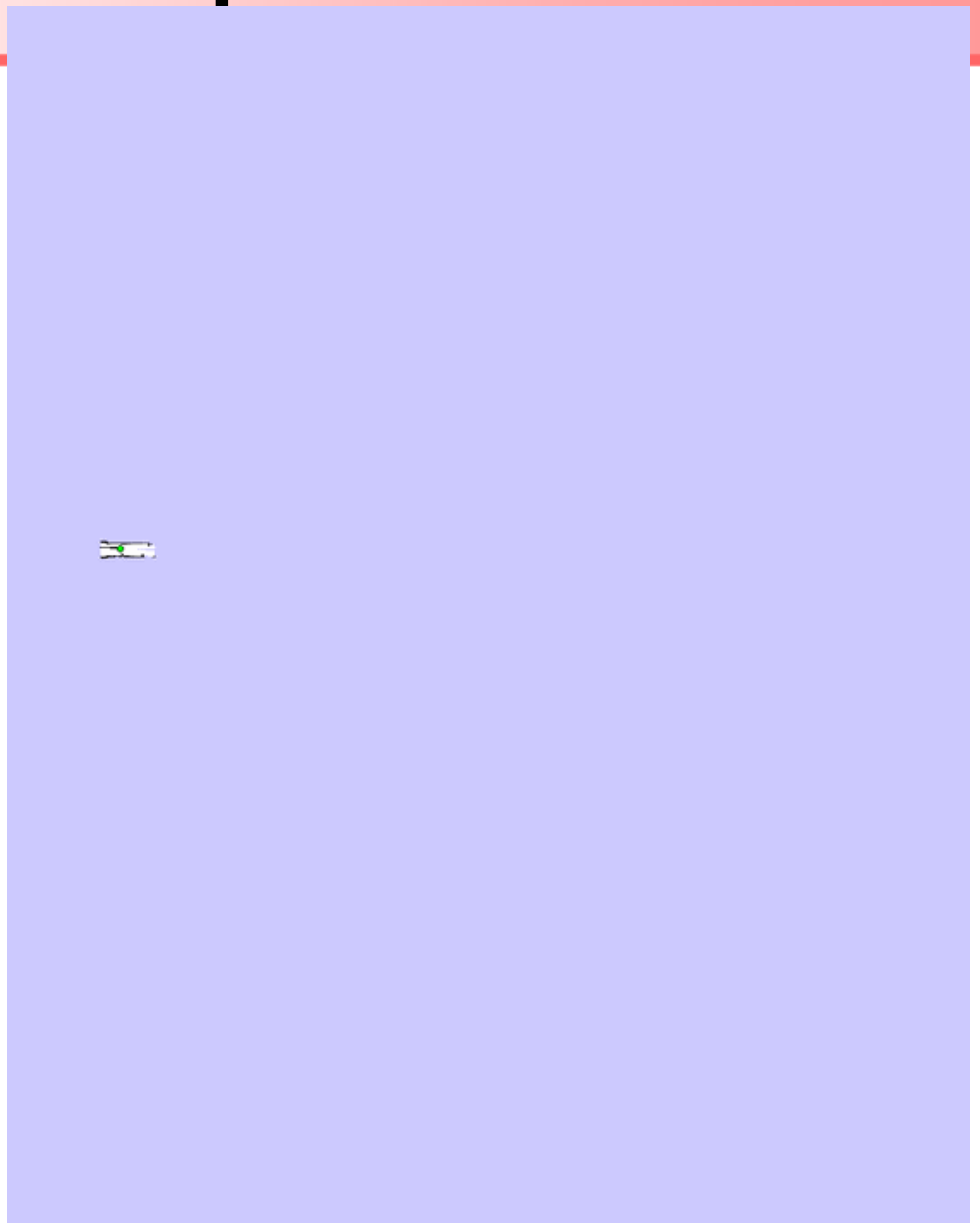
Loop Closure

- FastSLAM may perform inferior to EKF in loop closure,
- Larger particle sets perform better,
- Preserving diversity of particles critical



Another Example

- Mapping of Bruceton mine



More on FastSLAM

- M. Montemerlo, S. Thrun, D. Koller, and B. Wegbreit. FastSLAM: A factored solution to simultaneous localization and mapping, *AAAI02*
- D. Haehnel, W. Burgard, D. Fox, and S. Thrun. An efficient FastSLAM algorithm for generating maps of large-scale cyclic environments from raw laser range measurements, IROS03
- M. Montemerlo, S. Thrun, D. Koller, B. Wegbreit. FastSLAM 2.0: An Improved particle filtering algorithm for simultaneous localization and mapping that provably converges. IJCAI-2003
- G. Grisetti, C. Stachniss, and W. Burgard. Improving grid-based slam with rao-blackwellized particle filters by adaptive proposals and selective resampling, ICRA05
- A. Eliazar and R. Parr. DP-SLAM: Fast, robust simultanous localization and mapping without predetermined landmarks, IJCAI03

Questions?…